OPERATING SYSTEMS LAB REPORT

Assignment 10: Memory Management & Paging

Environment: xv6-public (x86)
Architecture: x86_64-linux-gnu / QEMU
Date: November 2026

This report documents the implementation, kernel source modifications, user-level test programs, and analysis for four memory management tasks in the xv6 operating system: tracking physical allocation, virtual-to-physical address translation, page table protection, and page replacement algorithm simulation.

1. PHYSICAL MEMORY ALLOCATION TRACKING

Objective: Track the total number of physical memory pages allocated and freed by the kernel, and expose these metrics to user space via a system call.

Kernel Modifications:

- `kalloc.c`: Added global counter variables `total_allocated_pages` and `total_freed_pages`. Incremented allocation counter inside `kalloc()` and freed counter inside `kfree()`. Created `dump_memstats()` to traverse the `kmem.freelist` under lock protection to count currently free physical pages.
- `sysproc.c` & `defs.h`: Defined `sys_memstats(void)` wrapper calling `dump_memstats()`. Added header prototype in `defs.h`.
- `syscall.h`, `syscall.c`, `user.h`, `usys.S`: Assigned syscall ID `SYS_memstats (22)`, registered in kernel dispatch vector, and exposed `int memstats(void)` interface.

User Program (`memtest.c`) & Execution Observation:

```
Initial Memory Stats: Total Allocated: 441 | Total Freed: 2 | Available Pages: 3657 Allocatin
g memory using sbrk(4096 * 5)... Memory Stats After Allocation: Total Allocated: 446 | Total
Freed: 2 | Available Pages: 3652 Freeing memory sbrk(-(4096 * 5))... Memory Stats After Freei
ng: Total Allocated: 446 | Total Freed: 7 | Available Pages: 3657
```

Observation: Calling `sbrk(4096 * 5)` allocated exactly 5 pages (increasing total allocations from 441 to 446 and reducing available pages from 3657 to 3652). Deallocating via negative `sbrk` incremented freed pages to 7 and restored physical available pages to 3657.

2. VIRTUAL-TO-PHYSICAL ADDRESS TRANSLATION

Objective: Implement a system call `int vtop(int virtual_address)` that translates a user virtual address into its underlying physical frame address.

Kernel Modifications:

- `vm.c`: Implemented `vtop_impl(uint va)`. It retrieves the current process page directory (`myproc()->pgdir`) and invokes internal helper `walkpgdir()` to look up the Page Table Entry (`pte_t*`).
- **Address Mathematics:** Verified that entry exists and `*pte & PTE_P` (Present bit) is set. Extracted high physical frame address via `PTE_ADDR(*pte)` and combined it with low 12-bit offset via `(va & 0xFFF)`: `pa = PTE_ADDR(*pte) | (va & 0xFFF);`
- `syscall.h`, `syscall.c`, `sysproc.c`, `user.h`, `usys.S`: Registered syscall ID `SYS_vtop (23)` using `argint(0, &va)` argument decoding. Unmapped/invalid addresses safely return `-1`.

User Program (`vttest.c`) & Execution Output:

```
Stack Var -> VA: 0x2FCC | PA: 0xDEDFFCC sbrk Heap -> VA: 0x3000 | PA: 0xDFBC000 Invalid VA -> VA: 0x7FFFFFFF | Result: -1 (Unmapped)
```

Observation: Virtual addresses mapped to physical RAM frames sharing identical 12-bit page offsets (`0x2FCC` mapped to frame base `0xDEDF000` + offset `0xCC`). Unmapped virtual address `0x7FFFFFFF` correctly returned `-1` without kernel crash.

3. PAGE TABLE MANAGEMENT AND PAGE PROTECTION

Objective: Inspect page table entry flags (P, W, U) and modify page write permissions dynamically to enforce read-only protection.

Kernel Modifications:

- `vm.c` : Implemented `pageinfo_impl(uint va)` to extract and log PFN (`PTE_ADDR(*pte) >> 12`) and flags Present (`PTE_P`), Writable (`PTE_W`), and User (`PTE_U`).
- `vm.c` : Implemented `setreadonly_impl(uint va)` to strip write permissions: `*pte &= ~PTE_W;` . Crucially, reloaded CR3 register via `lcr3(V2P(pgdir))` to immediately flush the CPU Translation Lookaside Buffer (TLB).
- Syscall dispatch entries created for `SYS_pageinfo (24)` and `SYS_setreadonly (25)` .

User Program (`pagetest.c`) & Page Fault Demonstration:

```
Allocated 1 page at VA: 0x3000 Page Info BEFORE setreadonly: VA: 0x3000 | Frame (PFN): 0xdfbc | Flags: P=1 W=1 U=1 Calling setreadonly()... Page Info AFTER setreadonly: VA: 0x3000 | Frame (PFN): 0xdfbc | Flags: P=1 W=0 U=1 Attempting to write to read-only page (Triggers Trap)... pid 3 pagetest: trap 14 err 7 on cpu 0 eip 0x8f addr 0x3000--kill proc
```

Observation: Clearing `PTE_W` modified page flags from `W=1` to `W=0` . The subsequent memory write `ptr[0] = 'B'` triggered hardware **Trap 14 (Page Fault Exception)** with error code 7 (user write permission violation), causing the kernel to terminate the process cleanly.

4. PAGE REPLACEMENT ALGORITHMS SIMULATION

Objective: Implement and evaluate First-In First-Out (FIFO), Least Recently Used (LRU), and Optimal (OPT) page replacement algorithms across varying frame capacities (3, 4, 5 frames) using reference string: `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` (20 references).

Algorithm	Frames	Page Faults	Page Hits	Hit Ratio (%)	Fault Ratio (%)
FIFO	3	15	5	25%	75%
LRU	3	12	8	40%	60%
Optimal	**3**	**9**	**11**	**55%**	**45%**
FIFO	4	10	10	50%	50%
LRU	4	8	12	60%	40%
Optimal	**4**	**8**	**12**	**60%**	**40%**
FIFO	5	9	11	55%	45%
LRU	5	7	13	65%	35%
Optimal	**5**	**7**	**13**	**65%**	**35%**

Comparative Analysis & Findings:

- **Optimal Algorithm:** Achieves the lowest fault count (9 faults at 3 frames, hit ratio 55%) because it leverages future knowledge of the reference sequence to evict the page that will not be accessed for the longest time.
- **LRU Algorithm:** Outperforms FIFO significantly (12 faults vs 15 faults at 3 frames). By tracking temporal locality (most recent access timestamp), LRU closely approximates Optimal performance as frame capacity increases (matching Optimal at 4 and 5 frames with 8 and 7 faults respectively).
- **FIFO Algorithm:** Demonstrates the highest page fault count across all configurations due to its rigid queue-based eviction policy, which replaces pages based strictly on load order regardless of access frequency.

5. CONCLUSION

The assignment successfully extended xv6's memory subsystem. Physical tracking provided real-time allocator introspection; virtual-to-physical translation validated hardware two-level paging mechanics; page protection demonstrated kernel-enforced hardware trap protection via TLB flushes; and page replacement simulation highlighted temporal locality advantages in virtual memory management.